

SQL SP Harness — VS Code Extension

Technical Design Document

Field	Value
Project	vscode-sql-sp-harness
Extension ID	DeeprajAdhikary.sql-sp-harness
Version	0.0.1
Backend	sql-sp-harness (PyPI)
License	MIT
Last updated	2026-06-06

1. Purpose and scope

1.1 Problem statement

Developers who edit T-SQL stored procedures in VS Code or Cursor need a safe way to **analyze** procedure structure and **generate debug harness scripts** without manually commenting out DML, adding `PRINT` traces, or switching to SSMS. Enterprise SQL Server environments often lack live step-debugging inside stored procedures.

1.2 Solution

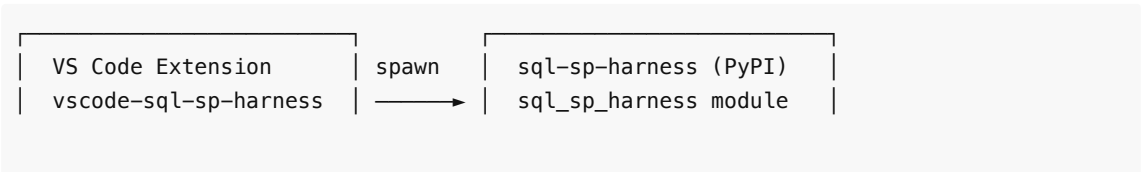
This extension is a **thin VS Code shell** around the published Python package [sql-sp-harness](#) . It:

- Invokes `python -m sql_sp_harness analyze|generate` on the user's SQL source
- Presents results in native VS Code UI (webview panel, output channel, status bar)
- Automates backend setup (`pip install`) when Python is available

1.3 Out of scope

Item	Rationale
Live debugging / breakpoints	Backend generates static scripts only
SQL parsing in TypeScript	All transform logic lives in Python
Bundled Python runtime (v1)	Planned future enhancement; v1 requires system Python
Marketplace publish automation	Separate release process

1.4 Relationship to sql-sp-harness



TypeScript UI layer

analyze / generate CLI

The extension **does not vendor** Python source. It depends on the PyPI package at runtime.

2. Architecture overview

2.1 Layered design

VS Code / Cursor	
Commands & Menus	extension.ts (activation hub)
Context submenu	generateDebugScript()
Command palette	runAnalyze()
Status bar	verifySetup()
UI Components	analyzeReportPanel.ts (webview) configureSettings.ts (QuickPick) BackendStatusBar (status bar)
Integration	spDebugBackend.ts backendSetup.ts cliLog.ts
External process	python -m sql_sp_harness ...

2.2 Module responsibilities

Module	Responsibility
extension.ts	Activation, command registration, orchestrate analyze/generate flows
spDebugBackend.ts	Python discovery, import sql_sp_harness probe, CLI spawn
backendSetup.ts	Startup auto-install, status bar lifecycle, pip install
analyzeReportParser.ts	Parse plain-text CLI analyze output into structured rows
analyzeReportPanel.ts	Tabbed webview (Summary / Warnings / Identified), save report, generate from panel
configureSettings.ts	Interactive Cmd+Shift+P settings wizard
cliLog.ts	Capture --log-file output into Output channel / disk
sqlSource.ts	Shared type for SQL source context passed between commands

2.3 Activation model

- activationEvents** : [] — extension activates on startup (* implicit in VS Code 1.74+)

- On activate: create output channel, status bar, run `runStartupBackendSetup()` asynchronously
- No workspace folder required; commands operate on active editor or passed `Uri`

3. Data flows

3.1 Generate debug script

```
User → spDebug.generate (uri?)
  → requireBackend()           probe Python + sql_sp_harness
  → resolveSqlSource()         file or selection text
  → write temp input.sql
  → runSpDebugCli(["generate", "-i", ..., "-o", ..., "--trace-style", ...])
  → optional --log-file, --quiet
  → read output_debug.sql
  → open untitled SQL document
  → append step log to Output channel
```

Temp directory: `os.tmpdir()/sql-sp-harness-*` — input, output, and log files are ephemeral unless `spDebug.saveLogFile` copies the log beside the source `.sql`.

3.2 Analyze procedure

```
User → spDebug.analyze (uri?)
  → requireBackend()
  → resolveSqlSource()
  → runSpDebugCli(["analyze", "-i", ..., "--plain"])
  → optional --log-file
  → parse stdout → analyzeReportParser
  → showAnalyzeReportPanel (webview with tabs)
  → full report also logged to Output channel
```

The webview receives the **same SQL** that was analyzed via `SqlSourceContext`, enabling **Generate Debug Script** from the panel without re-reading the file (important for selection-based analysis).

3.3 Startup backend setup

```
activate()
  → BackendStatusBar.setChecking()
  → resolveSpDebugBackend()
    └─ OK → status bar Ready
    └─ Python missing → status bar error + first-run notice
    └─ Python OK, package missing
      → if spDebug.autoInstallBackend
        → pip install sql-sp-harness (with progress notification)
        → re-probe
      → else → status bar Setup required
```

4. Backend integration

4.1 Python discovery

Platform	Candidates (in order)
macOS / Linux	python3, python
Windows	python, python3, py

Override with `spDebug.pythonPath` .

4.2 Backend probe

```
import sql_sp_harness; print(sql_sp_harness.__version__)
```

10-second timeout per candidate. First successful import wins.

4.3 CLI invocation

```
python -m sql_sp_harness analyze -i <input> --plain [--log-file <path>]
python -m sql_sp_harness generate -i <input> -o <output> --trace-style
print|raiserror [--log-file] [--quiet]
```

Spawn uses `child_process.spawn` with `shell: false` . Windows `py` launcher gets `-3` prefix.

4.4 Exit codes

Code	Meaning	Extension behavior
0	Success	Normal completion
2	Success with warnings (generate)	Show warning toast; still open output
Other	Failure	Error toast; log stderr/stdout

5. User interface

5.1 Commands

Command ID	Title	Entry points
<code>spDebug.generate</code>	Generate Debug Script	Submenu, palette
<code>spDebug.analyze</code>	Analyze Procedure	Submenu, palette
<code>spDebug.verifySetup</code>	Verify Python Setup	Submenu, palette, status bar click
<code>spDebug.configure</code>	Configure Settings	Submenu, palette
<code>spDebug.openSettings</code>	Open Extension Settings	Submenu, palette
<code>spDebug.generateAnalyzed</code>	<i>(internal)</i>	Analysis panel button

5.2 Context submenu

Right-click on `.sql` files (Explorer, editor, tab title) shows **SQL SP Harness** ▸ with grouped items:

- 1. **Run** — Generate, Analyze
- 2. **Setup** — Verify, Configure, Open Settings

Implemented via `contributes.submenus` in `package.json`.

5.3 Analysis webview panel

- **View type:** `sqlSpHarness.analyze`
- **Column:** `ViewColumn.Beside`
- **Tabs:** Summary (counts), Warnings, Identified (line-level detail)
- **Actions:** Save Report (plain-text via save dialog), Generate Debug Script
- **Navigation:** Click line reference → `gotoLine` in source editor
- **Reuse:** Single panel instance updated on subsequent analyze calls

5.4 Output channel

- **Name:** `SQL SP Harness`
- Singleton instance created at activation (fixes duplicate channel bug)
- Receives: startup logs, CLI stdout/stderr, step logs (`--- Step log ---` section)

5.5 Status bar

- **Alignment:** Right, priority 100
- **States:** Checking → Installing → Ready | Python not found | Setup required
- **Click action:** `spDebug.verifySetup`

6. Configuration

All settings use prefix `spDebug.*` and are grouped in Settings UI:

Section	Keys
SQL SP Harness	<code>pythonPath</code> , <code>pipPackage</code> , <code>autoInstallBackend</code>
SQL SP Harness ▸ Generate	<code>traceStyle</code>
SQL SP Harness ▸ Logging	<code>logToOutput</code> , <code>saveLogFile</code> , <code>quietWhenLogging</code>

Settings filter: `@ext:DeeprajAdhikary.sql-sp-harness`

Interactive configuration: **SQL SP Harness: Configure Settings** (QuickPick loop with section separators).

7. Security and safety

Concern	Mitigation
Arbitrary command execution	Only spawns configured Python + fixed module args
User SQL content	Written to temp files locally; never sent to network
Webview XSS	HTML escaped in <code>analyzeReportPanel</code> ; CSP restricts scripts

Production misuse	Generated scripts include backend banner; docs warn dev-only
pip install on activate	User-controlled via autoInstallBackend; uses configured pipPackage

8. Error handling

Scenario	UX
Backend not found	Error toast, copy pip command, open settings
Python timeout	Try next candidate
pip install fails	Status bar warning, output log, manual instructions
CLI non-zero exit	Error toast pointing to Output channel
No .sql file / wrong language	Warning toast
Untitled analyze + generate	Works via in-memory SqlSourceContext; save log file disabled without path

9. Build and distribution

9.1 Development

```
npm install && npm run compile
# F5 → Extension Development Host
```

9.2 VSIX packaging

```
npm run package # → dist/sql-sp-harness.vsix
```

Uses `@vscode/vsce@2.32.0`. TypeScript compiles to `out/`.

9.3 Documentation site

Static site in `/docs` — GitHub Pages from `main` branch `/docs` folder.

```
npm run docs:build # compile SCSS
npm run docs:pdf # TECHNICAL_DESIGN.md → PDF
```

10. Testing strategy

Area	Approach
TypeScript compile	<code>tsc --strict</code> in CI

Parser	Manual verification against sample CLI output
Extension E2E	F5 Extension Development Host + sample .sql in /samples
Backend	Relies on sql-sp-harness project's own pytest suite

Recommended manual smoke test:

1. Verify Python Setup
2. Analyze `samples/enterprise_complex_proc.sql`
3. Generate from analysis panel
4. Confirm step log in Output channel
5. Configure settings via QuickPick

11. Future enhancements

Enhancement	Priority	Notes
Bundled platform binaries	High	Eliminate Python/pip requirement for Marketplace users
Marketplace publish CI	Medium	vsce publish on tag
--encoding, --keep-comments settings	Low	Expose more CLI flags
Tree view of identified statements	Low	Alternative to webview
Workspace trust prompt before pip	Medium	Enterprise policy alignment

12. Repository layout

```

vscode-sql-sp-harness/
├── src/
│   ├── extension.ts           # Entry point, commands
│   ├── spDebugBackend.ts      # Python / CLI integration
│   ├── backendSetup.ts        # Auto-install, status bar
│   ├── analyzeReportPanel.ts  # Webview UI
│   ├── analyzeReportParser.ts # Report parser
│   ├── configureSettings.ts   # Settings wizard
│   ├── cliLog.ts              # Log file handling
│   └── sqlSource.ts            # Shared types
├── docs/                       # GitHub Pages site + TDD
├── samples/                    # Example .sql procedures
├── images/                     # Extension icon
├── package.json                # Extension manifest
└── .github/workflows/          # CI + Pages deploy

```

13. References

- Backend docs: <https://deeprajdeveloper.github.io/sql-sp-harness/>
- Backend repo: <https://github.com/DeeprajDeveloper/sql-sp-harness>
- VS Code Extension API: <https://code.visualstudio.com/api>
- Extension docs site: <https://deeprajdeveloper.github.io/vscode-sql-sp-harness/>

Document generated for the vscode-sql-sp-harness project. For user-facing guides see the [documentation site](#).